

LocalStack services live in isolation as separate processes available via HTTP, which fosters true decoupling and more closely resembles the real cloud environment.

Pluggable services: All services in LocalStack are easily pluggable and replaceable, due to the fact that we are using isolated processes for each service. This allows us to keep the framework up-to-date and select best-of-breed mocks for each individual service.

Installing LocalStack

The easiest way to install LocalStack is via Pip:

```
pip install localstack
```

Once installed, run the infrastructure using the following command:

```
localstack start
```

 **Note:** Please do not use sudo or the root user. LocalStack should be installed and started entirely under a local non-root user. If you have problems with permissions in MacOS X Sierra, install with 'pip install --user localstack'.

Running LocalStack in Docker

You can also spin up LocalStack in Docker, as follows:

```
localstack start --docker
```

Or by using *docker-compose* (you need to clone the repository first; it currently requires docker-compose version 2.1+):

```
docker-compose up
```

 **Note:** On MacOS you may have to run `TMPDIR=/private$TMPDIR localstack start --docker` if `$TMPDIR` contains a symbolic link that cannot be mounted by Docker.

Use an existing *docker-compose* project. Add in existing services. The project can be found in Docker Hub, so there's no need to download or clone the source:

```
version: '2.1'
services:
...
  localstack:
    image: localstack/localstack
    ports:
      - "4567-4584:4567-4584"
      - "${PORT_WEB_UI-8080}:${PORT_WEB_UI-8080}"
    environment:
      - SERVICES=${SERVICES- }
```

```
    - DEBUG=${DEBUG- }
    - DATA_DIR=${DATA_DIR- }
    - PORT_WEB_UI=${PORT_WEB_UI- }
    - LAMBDA_EXECUTOR=${LAMBDA_EXECUTOR- }
    - KINESIS_ERROR_PROBABILITY=${KINESIS_ERROR_PROBABILITY- }
  }
  - DOCKER_HOST=unix:///var/run/docker.sock
  volumes:
    - "${TMPDIR:-/tmp/localstack}:/tmp/localstack"
```

To facilitate interoperability, configuration variables can be prefixed with *LOCALSTACK_* in Docker. For instance, setting *LOCALSTACK_SERVICES=s3* is equivalent to *SERVICES=s3*.

LocalStack spins up the following core Cloud APIs on your local machine:

```
API Gateway at http://localhost:4567
Kinesis at http://localhost:4568
DynamoDB at http://localhost:4569
DynamoDB Streams at http://localhost:4570
Elasticsearch at http://localhost:4571
S3 at http://localhost:4572
Firehose at http://localhost:4573
Lambda at http://localhost:4574
SNS at http://localhost:4575
SQS at http://localhost:4576
Redshift at http://localhost:4577
ES (Elasticsearch Service) at http://localhost:4578
SES at http://localhost:4579
Route53 at http://localhost:4580
CloudFormation at http://localhost:4581
CloudWatch at http://localhost:4582
SSM at http://localhost:4583
SecretsManager at http://localhost:4584
StepFunctions at http://localhost:4585
CloudWatch Logs at http://localhost:4586
STS at http://localhost:4592
IAM at http://localhost:4593
```

Additionally, LocalStack provides a powerful set of tools to interact with the cloud services, including a fully featured KCL Kinesis client with Python binding, simple setup/teardown integration for nose tests, as well as an environment abstraction that allows easy switching between local and remote cloud implementations. 

By: Swapnil Kulkarni

The author is an open source enthusiast with experience in blockchain, cloud native solutions, containers and enterprise software product architectures. He has diverse experience with different private and hybrid cloud architectures with Amazon Web Services, etc. He is a technology hobbyist and writes at cloudnativetech.wordpress.com.